

Orin NX Quick Start

Get logged in, see the camera, talk to the flight controller. For participants who have just been handed a drone with a Jetson Orin NX on it.

COMPUTE	CARRIER	CAMERA	AUTOPILOT	REACH IT AT
Orin NX 16 GB	Seed A603	Sony IMX477	Betaflight / MSP	dcl@192.168.55.1

Who this is for: challenge participants who have just been handed a drone with a Jetson Orin NX on it, and want to be logged in and looking at camera frames within the hour.

You do not need to know anything about Jetson, device trees or L4T to get through this document. The companion document, `SOFTWARE-GUIDE.md`, explains every program in detail once you are in.

1. What is on the drone

Compute	NVIDIA Jetson Orin NX 16 GB, on a Sseed reComputer A603 carrier
OS	Ubuntu 22.04 (JetPack 6.2 / L4T r36.4.3), on an NVMe SSD
Camera	Sony IMX477 on the carrier's single 15-pin CSI-2 connector
Flight controller	Betaflight, wired to the Jetson's UART, speaks MSP
Login	user <code>dcl</code> , password <code>dcl</code>
Your tools	<code>~/target/</code> — already on the board, nothing to install
Power budget	25 W nvpmodel profile by default

Three things are worth internalising before you start, because each one has bitten somebody already:

1. **The camera is a raw Bayer sensor, not a webcam.** Frames go through NVIDIA's ISP (Argus), and Argus needs a graphics context — which a plain SSH session does not have. §5 tells you what to do about it.
2. **The flight controller speaks MSP, not MAVLink.** Betaflight is not ArduPilot. Tools that expect MAVLink will not work against it.
3. **The Jetson has no authority over the motors unless you give it some.** One program in `~/target` can arm the aircraft. It is clearly marked, in this document and in its own `--help`. Everything else is read-only.

2. Get logged in (5 minutes)

The board presents itself to your laptop as a USB network adapter. Plug a **USB-C to micro-USB** cable from your laptop into the board's micro-USB port — that is the small port next to the barrel jack, not one of the full-size USB-A ports.

```
ssh dcl@192.168.55.1      # password: dcl
```

That is the whole procedure when it works. `192.168.55.1` is fixed — the board always answers on it over USB, regardless of what Wi-Fi or Ethernet is doing.

If the SSH command hangs or says "no route to host", your laptop has not configured the new interface yet. Find it and give it an address by hand:

```
ip -br link                                # look for usb0, or enx<12 hex digits>
sudo ip addr add 192.168.55.100/24 dev <iface> # Linux
ping -c3 192.168.55.1
```

On macOS the interface appears in *System Settings* → *Network* as an RNDIS/NCM device; set it to DHCP and it will pick up `192.168.55.100` on its own. On Windows, expect an "NVIDIA USB Ethernet/RNDIS" adapter in *Network Connections*.

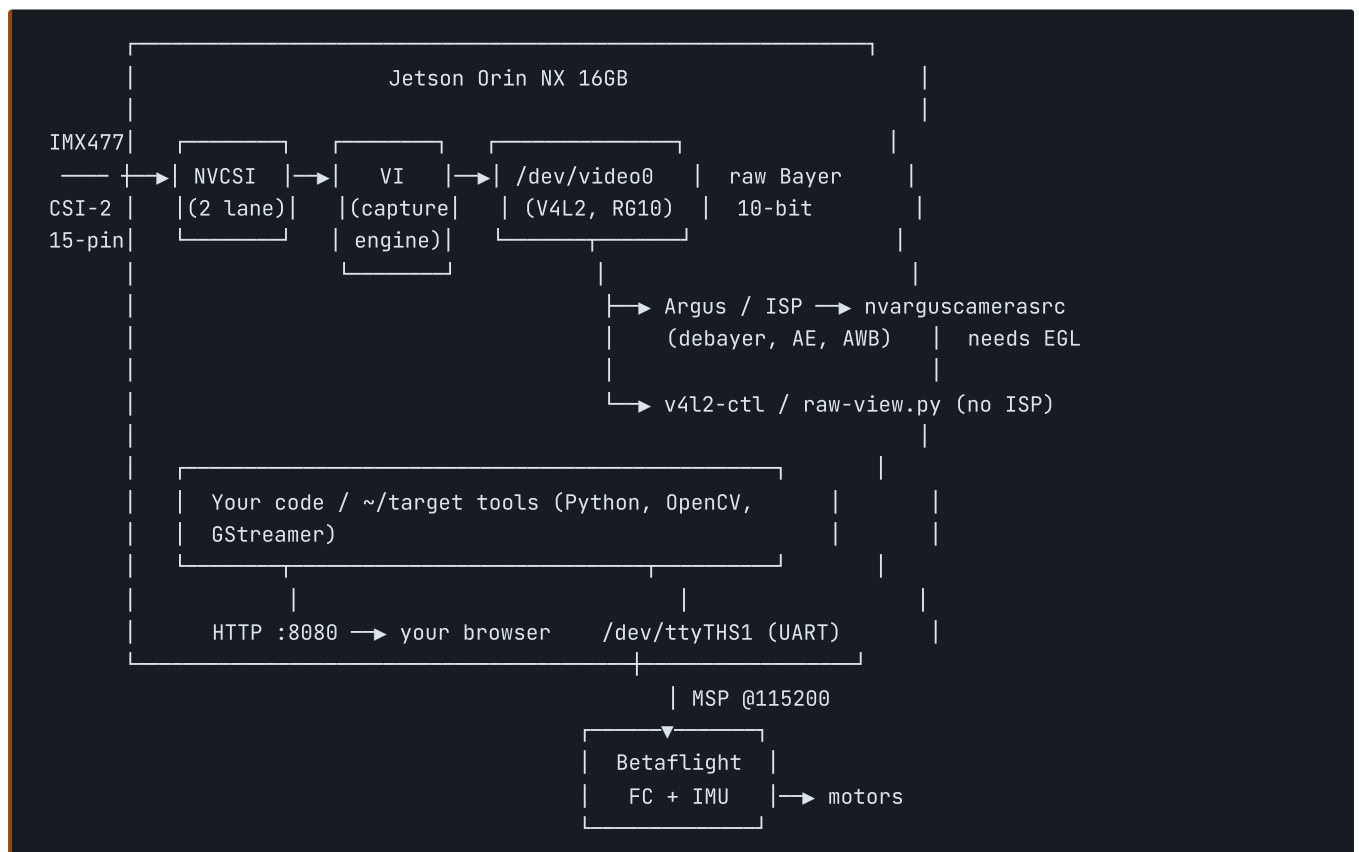
Still nothing? The same USB cable also gives you a serial console at `/dev/ttyACM0` (Linux/macOS) or a COM port (Windows) — `screen /dev/ttyACM0 115200` or PuTTY at 115200 baud gets you a login prompt even when the network half of the gadget is not up. If neither works, the board's USB device mode did not start; get in over Ethernet or hand the board back to the organisers, who have a diagnostic for exactly this (`~/target/usb-device-mode.sh`).

First five commands

```
uname -a                # kernel — expect 5.15.148-tegra
cat /etc/nv_tegra_release # L4T version — expect R36 (release), REVISION: 4.3
nvpmodel -q             # power mode — expect 25W
ls /dev/video0           # the camera node must exist
ls ~/target              # your toolbox
```

If `/dev/video0` is missing, stop and read §7 — nothing camera-related will work until it is back.

3. How the pieces fit together



Read that as three independent subsystems:

- **The camera path** ends at `/dev/video0`. Everything above it — Argus, ISP, GStreamer, OpenCV — is software you can bypass. When you are debugging, always ask "does `/dev/video0` produce bytes?" before asking anything else.
- **The flight-controller path** is one serial port. The Jetson polls the FC over MSP; the FC does not push. Nothing on this link is fast or deterministic enough to fly on — it is telemetry and supervision.
- **Your code** sits between them. The two paths share no clock and no hardware trigger. That is the single most important architectural fact on this board, and §6 explains why.

4. Your toolbox

Everything lives in `~/target`. Nothing here needs `pip`; every dependency is already in the image.

```
~/target/
├─ live-view.py           camera → browser, works over SSH      ← start here
├─ live-view-pts.py      same, with hardware capture timestamps
├─ live-view-imu.py      camera + live Betaflight attitude, one page
├─ show-camera.py        camera in a window on the board's own display
├─ raw-view.py           raw Bayer, ISP bypassed — "is the sensor alive?"
├─ frame-timestamps.py   per-frame timing and jitter, CSV out
├─ signoff.sh            the factory acceptance test (produces the PDF)
├─ bringup-check.sh      full stack check, stops at the first real failure
├─ camera-bind-check.sh  "why is the camera driver not bound?"
├─ usb-device-mode.sh    "why is 192.168.55.1 not answering?"
├─ msp/
│   ├─ setup_jetson_uart.sh  free the UART from the serial console
│   ├─ msp_bench.py          talk to the FC: info / telemetry / rc / demo
│   ├─ imu_check.py          functional IMU acceptance test
│   ├─ companion_listener_msp.py live telemetry table
│   └─ msp.py, msp_rc.py     the libraries your own code should import
```

5. See the camera

The one that works over SSH

```
~/target/live-view.py
```

It prints a URL. Open `http://192.168.55.1:8080/` in your browser. That is MJPEG over HTTP, so it needs nothing installed on your laptop.

Expect flat, washed-out colour over a plain SSH session, and read this before you file a bug about it. The IMX477 is a Bayer sensor: somebody has to debayer it and run auto-exposure and white balance. On this board that somebody is NVIDIA's Argus ISP, reached through `nvarguscamerasrc` — and Argus needs an EGL/graphics context that an SSH session does not have. It fails with `Failed to initialize EGLDisplay`, and `live-view.py` falls back to a software debayer with no AE and no AWB.

Two ways to get the real, ISP-processed image:

```
# (a) A desktop session is running on the board's own display. You need BOTH
#     the display number and that session's X authority file -- find the file
#     with `ls -a /run/user/1000/` or `ls ~/.Xauthority`, then:
DISPLAY=:0 XAUTHORITY=<that file> ~/target/live-view.py

# (b) You are sitting at the board with a monitor plugged in:
~/target/show-camera.py
```

Is the sensor actually alive?

When the picture is black, blank or absurd, go under the ISP entirely:

```
~/target/raw-view.py --ctrl exposure=20000 --ctrl gain=200
```

That reads raw Bayer straight from V4L2 — no ISP, no auto-exposure, nothing that can lie to you. There is no AE on this path, so you **must** set exposure and gain yourself; a dark first frame is expected, not a fault. Needs a display on the board.

Headless, the equivalent question is answered by:

```
v4l2-ctl -d /dev/video0 --list-formats-ext      # what modes does the sensor offer?
v4l2-ctl -d /dev/video0 --set-fmt-video=width=1920,height=1080,pixelformat=RG10 \
--stream-mmap --stream-count=100 --stream-to=/dev/null
```

If that captures 100 frames, the sensor, the CSI link and the capture engine are all fine and your problem is further up the stack.

6. Talk to the flight controller

Once per board, free the UART — by default Linux is using it as a serial console, and it will fight you for every byte:

```
~/target/msp/setup_jetson_uart.sh                # inspect only, changes nothing
sudo ~/target/msp/setup_jetson_uart.sh --apply # then do it
```

Then check the link:

```
python3 ~/target/msp/msp_bench.py --port /dev/ttyTHS1 info
```

You should get the firmware identity, which sensors the FC has detected, the battery voltage, and the list of reasons it is currently refusing to arm. `--port`, `--baud` and `--timeout` are **global** options: they go *before* the subcommand, not after it.

Live telemetry, still read-only:

```
python3 ~/target/msp/msp_bench.py --port /dev/ttyTHS1 telemetry --hz 20
```

Camera and attitude on one page, which is the most useful thing on the board for checking that both halves are live at the same instant:

```
~/target/live-view-imu.py --msp /dev/ttyTHS1
```

△ The one program that spins motors

```
python3 ~/target/msp/msp_bench.py --port /dev/ttyTHS1 demo --props-off
```

`demo` arms the aircraft and runs a throttle profile. It refuses to start without `--props-off`, which is your assertion that the propellers are off — the code cannot check it. Take the props off. Every other program in `~/target` either cannot arm the FC or actively holds it disarmed.

What this link is and is not

MSP is request/response over a 115200-baud serial line. You poll; the FC answers. Attitude at 30–50 Hz is realistic; a hard real-time control loop is not. Treat the FC link as **telemetry and supervision**, and keep anything that has to close a loop at flight rates inside Betaflight itself.

7. When something is wrong

Run the checker first. It walks the stack in dependency order and stops at the first genuine failure, so the last line it prints is the layer that actually broke:

```
sudo ~/target/bringup-check.sh
```

SYMPTOM	FIRST THING TO TRY
<code>ssh: no route to host</code> on 192.168.55.1	Configure the USB interface on your laptop (\$2); then the <code>/dev/ttyACM0</code> console
No <code>/dev/video0</code>	<code>sudo ~/target/camera-bind-check.sh</code> — it finds which link in the bind chain broke
Camera colours are flat / grey	Expected over SSH. Argus has no display. See \$5
<code>Failed to initialize EGLDisplay</code>	Same cause. <code>DISPLAY=:0</code> with a valid <code>XAUTHORITY</code> , or use <code>raw-view.py</code>
Black frames from <code>raw-view.py</code>	No auto-exposure on that path. Set <code>--ctrl exposure=...</code> and <code>--ctrl gain=...</code>
FC says nothing on <code>/dev/ttyTHS1</code>	<code>sudo ~/target/msp/setup_jetson_uart.sh --apply</code> , then check wiring and that the FC is powered
<code>Permission denied</code> on the serial port	Your user needs the <code>dialout</code> group — <code>dcl</code> already has it; log out and back in after any change
Board gets hot, then slow	Expected under sustained inference. <code>tegrastats</code> shows the thermal state; <code>nvpmodel -q</code> shows the budget
<code>import cv2</code> fails	You are in a virtualenv or installed OpenCV from pip. Use the system Python 3 — pip's OpenCV has no GStreamer

Two things you should **not** do:

- **Do not** `pip install opencv-python`. The image ships JetPack's OpenCV, built with GStreamer. The pip wheel is not, and `cv2.VideoCapture(pipeline, cv2.CAP_GSTREAMER)` will fail silently — you will get `False` from `isOpened()` and no explanation.
- **Do not try to change the device tree**. On this platform the booted device tree comes from the bootloader partition; writing `kernel-dtb` verifies clean and does nothing at all. Changing it means a full re-flash, which is the organisers' job.

8. Shutting down

The Jetson runs a real filesystem on a real SSD. Pulling the battery while it is writing will eventually corrupt it.

```
sudo shutdown -h now
```

Wait for the power LED to go out before disconnecting.

9. Where next

`SOFTWARE-GUIDE.md` covers every program in `~/target` in detail: all the options, what each one is really measuring, how to import the MSP library into your own code, and the camera/IMU timing constraints that matter if you are building anything visual-inertial.